# PASSWORD STRENGTH ANALYZER

Project Report

**IBM SkillsBuild Cyber Security BootCamp 2026**

Conducted by BharatCares

Submitted by: Dipanksha Elinje

October 2026

## 1. Abstract

Weak passwords remain one of the most common causes of account compromise. This project presents a Password Strength Analyzer, a Python tool that evaluates a password using entropy, rule-based checks and common-password detection. It produces a score out of 100, a strength rating and practical suggestions. The tool also includes a cryptographically secure password generator and a visual comparison of multiple passwords.

## 2. Introduction

Passwords are the first line of defence for most online accounts. Attackers use brute-force and dictionary attacks that try millions of guesses per second, so the length and unpredictability of a password strongly affect how long it takes to crack. Many users still choose short, predictable passwords such as "123456" or "password".

This project helps users understand how strong their password really is and how to improve it, using core cybersecurity concepts.

## 3. Problem Statement

Users often cannot tell whether a password is secure. Common composition rules (for example "add one number") are inconsistent and can give a false sense of safety: "Qwerty123" looks complex but is easily guessed. A tool is needed that measures strength objectively and explains how to fix weaknesses.

## 4. Objectives

- Measure password strength using entropy.
- Detect commonly used passwords and predictable patterns.
- Give a score, rating and clear improvement tips.
- Provide a secure password generator.
- Visualize and compare the strength of different passwords.

## 5. Technologies Used

| Technology | Purpose |
|---|---|
| Python 3 | Core programming language |
| Jupyter Notebook / Google Colab | Development and execution environment |
| pandas | Storing and displaying test results as a table |
| matplotlib | Bar chart visualization |
| re, math, string | Pattern matching, entropy calculation, character sets |
| secrets | Cryptographically secure random password generation |

# 6. Methodology

## 6.1 Entropy Calculation

Entropy measures unpredictability in bits. It is calculated as:

```
Entropy = Length x log2(Character Pool Size)
```

The character pool is built from the types present: lowercase (26), uppercase (26), digits (10) and symbols (32). A longer password with more character types has higher entropy.

## 6.2 Scoring

The final score (0-100) is the sum of: entropy points (up to 70), a length bonus (up to 15) and a variety bonus (up to 15). Penalties are then applied:

| Condition | Effect on score |
|---|---|
| Password is in the common-password list | Capped at 5 |
| Contains a sequence (abc, 123, cba) | -15 |
| Contains 3 or more repeated characters | -25 |

## 6.3 Rating Scale

| Score | Rating |
|---|---|
| 0 - 29 | Very Weak |
| 30 - 49 | Weak |
| 50 - 69 | Moderate |
| 70 - 84 | Strong |
| 85 - 100 | Very Strong |

## 6.4 Password Generator

The generator uses Python's secrets module, which draws from the operating system's secure random source. It guarantees at least one lowercase letter, uppercase letter, digit and symbol.

# 7. Code Explanation

- charset_size(password): returns the size of the character pool used by the password.
- calculate_entropy(password): computes entropy in bits.
- has_sequence(password): detects ascending or descending runs such as abc or 321.
- has_repeats(password): detects three or more identical characters in a row using a regular expression.
- analyze_password(password): combines all checks and returns the score, rating, entropy and feedback.
- generate_password(length): creates a secure random password.

## 8. Results

The tool was tested on ten passwords of varying quality:

| Password | Length | Entropy (bits) | Score | Rating |
|---|---|---|---|---|
| 123456 | 6 | 19.9 | 0 | Very Weak |
| password | 8 | 37.6 | 5 | Very Weak |
| abc123 | 6 | 31.0 | 0 | Very Weak |
| Qwerty123 | 9 | 53.6 | 0 | Very Weak |
| aaaaaaaa | 8 | 37.6 | 20 | Very Weak |
| hello2026 | 9 | 46.5 | 56 | Moderate |
| Hello@2026 | 10 | 65.5 | 80 | Strong |
| X9$kLm!2pQz#7vR | 15 | 98.3 | 85 | Very Strong |
| MyDog#Loves2Run | 15 | 98.3 | 100 | Very Strong |
| T!g3r-Blue-Moon-88 | 18 | 118.0 | 100 | Very Strong |

Observations:

- Common passwords and simple sequences scored near zero regardless of length.
- "Qwerty123" has mixed case and digits but is a well-known pattern, so it was rated Very Weak.
- Long passphrases with mixed character types reached the highest scores.

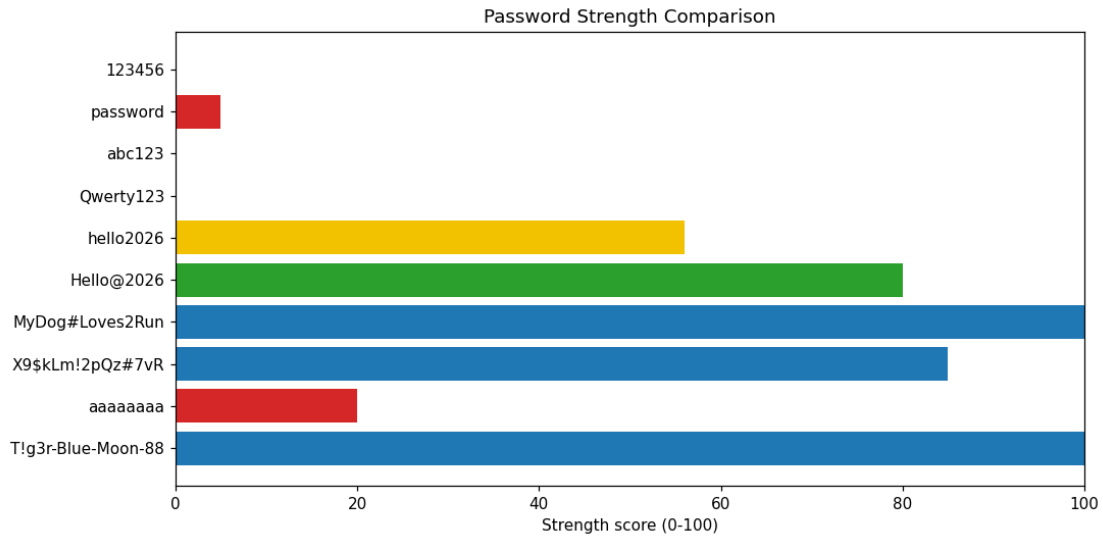



*Figure 1: Password strength comparison*

## 9. Conclusion

The project shows that password strength depends mainly on length, unpredictability and avoiding well-known patterns, not just on adding one number or symbol. The analyzer gives users a clear, explained assessment and a way to generate safer passwords.

## 10. Limitations and Future Scope

- The common-password list is small; a larger wordlist such as RockYou would improve detection.
- Dictionary words with simple substitutions (for example P@ssw0rd) are only partly detected.
- Future work: integrate the Have I Been Pwned API, build a Streamlit web interface, and train a machine-learning model on a leaked-password dataset.

## 11. References


- NIST Special Publication 800-63B: Digital Identity Guidelines (Authentication and Lifecycle Management).
- OWASP Authentication Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- Python documentation: secrets module: https://docs.python.org/3/library/secrets.html
- IBM SkillsBuild Cyber Security course material.